CEA / CMC ACADEMY · WEB APPLICATION VAPT

PHP2Shell — Exploitation Walkthrough

A runtime RCE that no application patch can fix

CVE-2026-109618 (memory disclosure) · CVE-2026-109619 (remote code execution)

> **Authorised use only.**
> This walkthrough exploits a deliberately vulnerable container on 127.0.0.1:8088 inside an isolated lab, using the public php2shell exploit chain against the lab's own loopback target. It is written to teach VAPT practitioners how the attack works and how to report and remediate it. Only run it against systems you are explicitly authorised to test. These CVEs are under active exploitation in the wild — patch first (final sections).

Learning Objectives

- Explain what session.upload_progress does and why it runs before any application code.
- Understand how a PHP memory bailout leaves a dangling pointer that survives into the next request.
- Distinguish the two CVEs: a memory-disclosure path and a remote-code-execution path.
- Run the public exploit chain to leak memory, defeat ASLR and execute a command.
- Detect the attack and apply the mitigations that actually work.

Worked answers to all five are consolidated in the final section.

Beneficiary

Written first for the people who exploit and report this on an engagement, and for the teams who then have to fix it:

- **VAPT practitioners / penetration testers** — the primary audience: fingerprint the stack, run the chain, prove impact, and write it up with accurate root cause and remediation.
- **Red teams** — a runtime RCE that lands against hardened, fully-patched applications, for realistic access scenarios in authorised operations.
- **System administrators & platform / container owners** — the only complete fix (patch the PHP runtime, restart workers) and the interim controls that buy time.
- **SOC analysts & detection engineers** — the Set-Cookie / oversized PHPSESSID and multipart-POST signatures to alert on.
- **Incident responders** — recognise a compromise that lives beneath the application and survives every app patch.

Narrative / Scenario

> **The engagement**
> You are running a web-application VAPT against a creative-studio site. The application looks ordinary — a polished marketing front end with no obvious upload, login or injection surface — and the client is confident because the app is fully patched and the code has been reviewed. During fingerprinting you notice the stack is **PHP 8.4.26** behind Apache, which is in range for PHP2Shell. Your task is to demonstrate that the application being clean is irrelevant: the vulnerability is in the PHP runtime underneath it. You fingerprint the target, run the public chain to prove remote code execution, retrieve the flag as proof of impact, and close with detection signatures and the remediation the client must apply.

Technical Background

The feature it stands on

When a browser uploads a large file, the server has no simple way to tell the page how far the upload has got. PHP solves this with session.upload_progress. While PHP is still reading a multipart/form-data body off the wire, it writes a running description of the upload into the user's $_SESSION. It is enabled by default and switched on by including a form field named PHP_SESSION_UPLOAD_PROGRESS.

The detail that matters is **when** this happens. Body parsing is one of the first things PHP does — long before it compiles and runs the target script. The progress updates are written from inside the body-parsing callbacks, **before any application code executes**. Reaching this code needs no login, no session_start(), and no application logic: any URL that accepts a POST is enough.

The flaw: a bailout that leaves a dangling pointer

When a request exceeds memory_limit, PHP performs a **bailout** — a longjmp() back to a recovery point that skips normal cleanup. If the out-of-memory condition strikes **after** a block is freed but **before** the pointer to it is cleared, the bailout leaves a dangling pointer behind — **CWE-824: Access of Uninitialised Pointer**.

Under Apache prefork with mod_php, one worker serves request after request without restarting, so the stale pointer persists into the next request. An attacker who controls what gets allocated into that freed space controls what the pointer now refers to. From this one defect come two CVEs:

CVE	Class	What it does
CVE-2026-109618	Memory disclosure	Stale pointer read back as an oversized string; PHP returns it in a response header, leaking worker memory.
CVE-2026-109619	Remote code execution	Turns the corrupted state into control over the program's flow.

The PHP changelog names the root cause precisely: *"Avoid state corruption via bailout"*.

Walkthrough

Proceed through the lab in this order, working from the machine against the target on http://127.0.0.1:8088/.

Step 1 — Fingerprint the stack

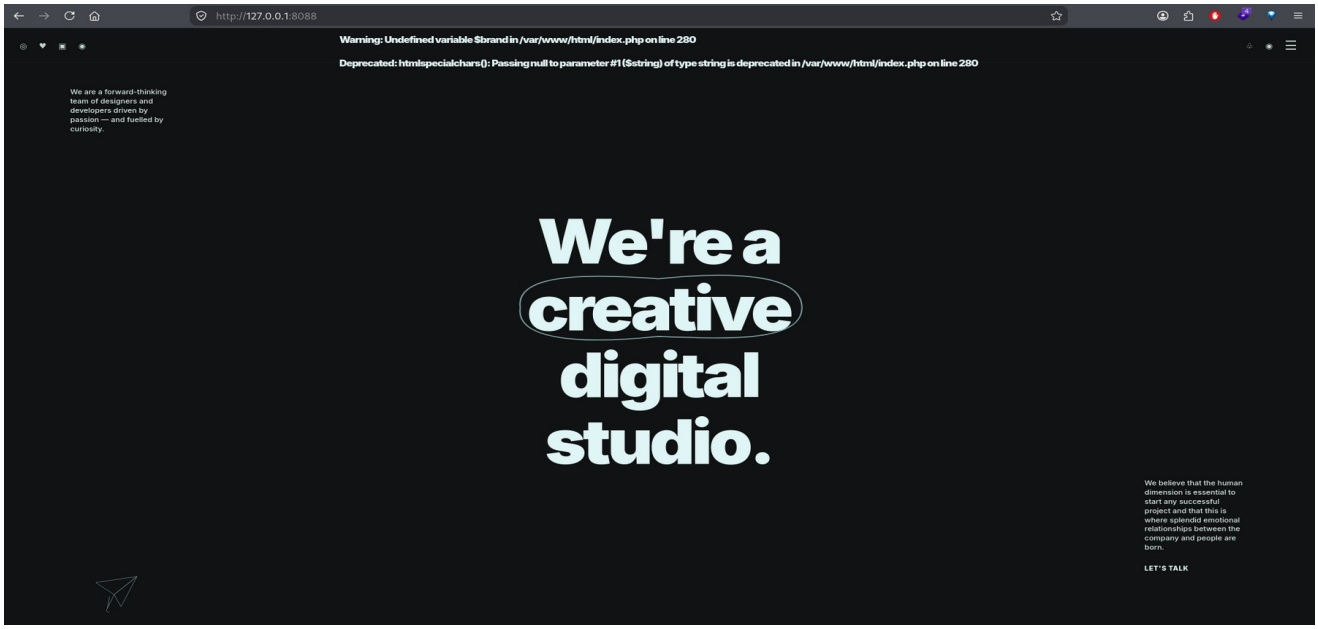
As an external assessor you fingerprint from the outside. A HEAD request exposes the stack in the response headers:

```
# curl -I http://127.0.0.1:8088/
HTTP/1.1 200 OK
Date: Fri, 06 Nov 2026 05:25:41 GMT
Server: Apache/2.4.69 (Debian)
X-Powered-By: PHP/8.4.26
Connection: close
Content-Type: text/html; charset=UTF-8
```

X-Powered-By: PHP/8.4.26 is the tell — one release short of the 8.4.27 fix, so it is in range for PHP2Shell.

Step 2 — Review the target

Browsing the site shows a complete creative-studio template, with PHP notices rendered inline on the page:



Target site — inline Warning / Deprecated notices leaking /var/www/html/index.php line 280.

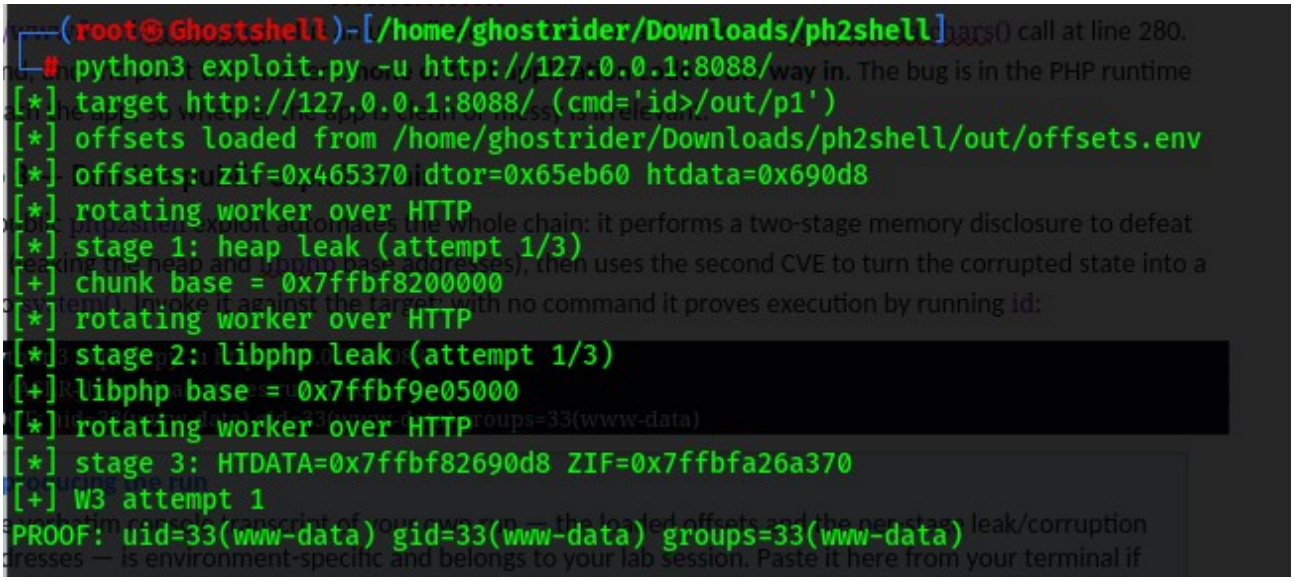
Two findings for the report. First, display_errors is enabled in production — full internal path disclosure (/var/www/html/index.php) plus an undefined variable and a deprecated htmlspecialchars() call at line 280. Second, and the point that matters: **none of that application code is the way in**. The bug is in the PHP runtime beneath the app, so whether the app is clean or messy is irrelevant.

Step 3 — Run the public exploit chain

The public php2shell exploit automates the whole chain: it performs a two-stage memory disclosure to defeat ASLR (leaking the heap and libphp base addresses), then uses the second CVE to turn the corrupted state into a call to system(). Invoke it against the target; with no command it proves execution by running id:

```
# python3 exploit.py -u http://127.0.0.1:8088/
   ... (ASLR-defeat leak stages run here) ...
PROOF: uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

> **Reproducing the run**
>
> The verbatim console transcript of your own run — the loaded offsets and the per-stage leak/corruption addresses — is environment-specific and belongs to your lab session. Paste it here from your terminal if your report needs the full trace: *[paste your console transcript]*



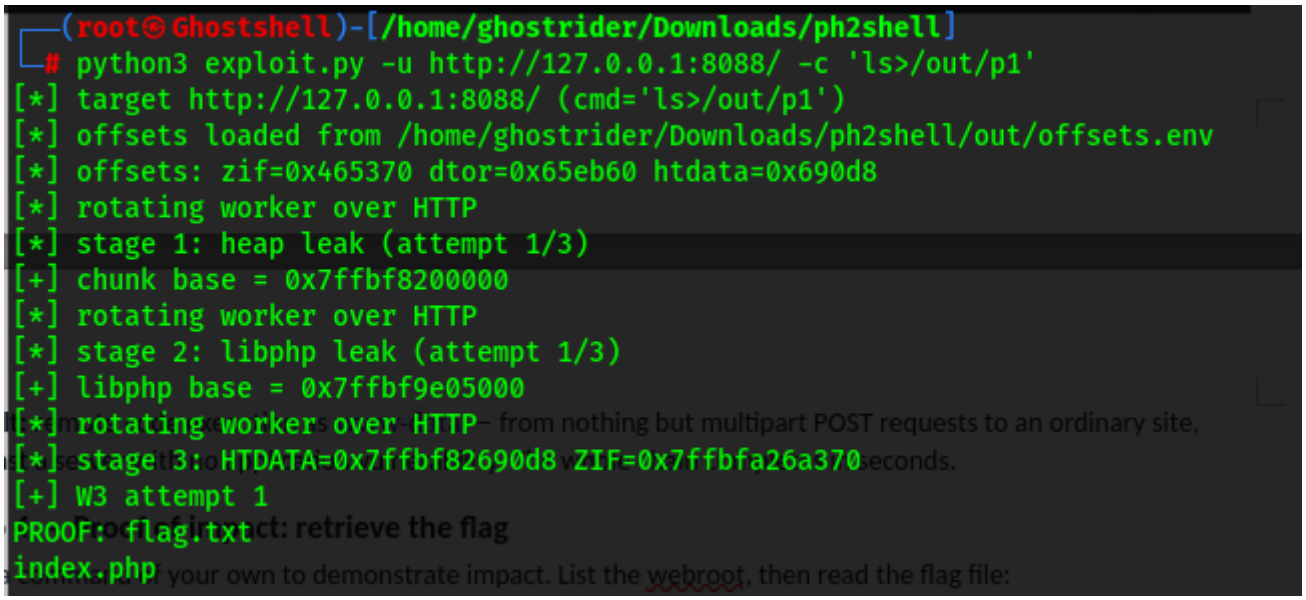
Result: remote code execution as www-data — from nothing but multipart POST requests to an ordinary site, against a server with no application vulnerability. The whole chain completes in seconds.

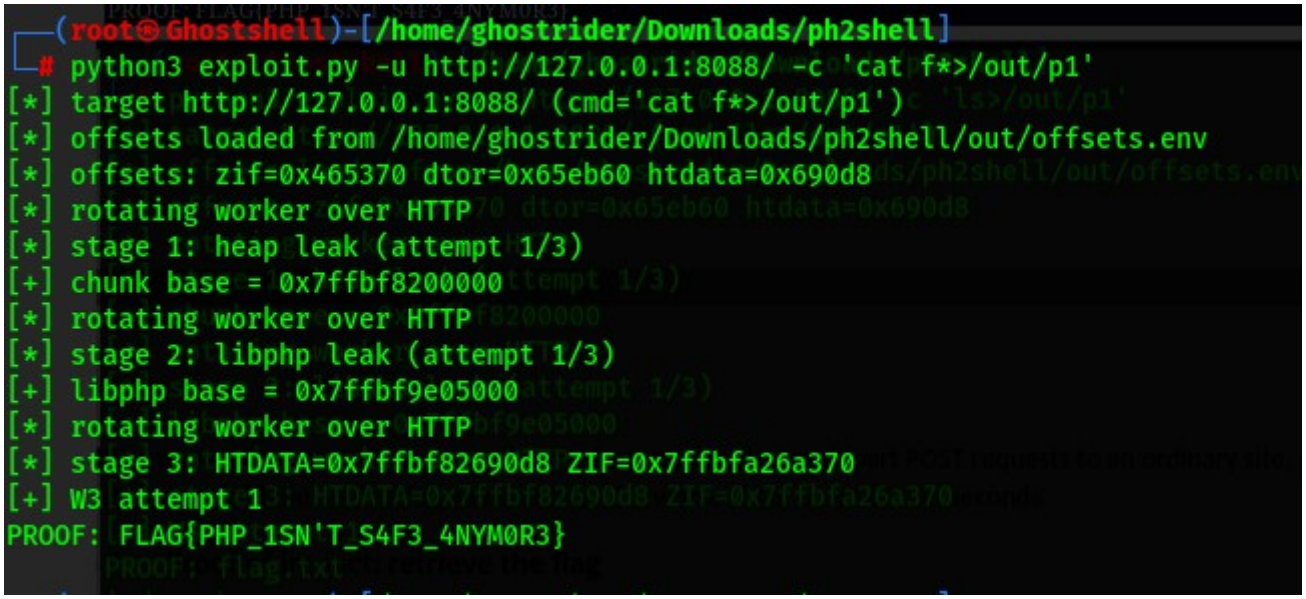
Step 4 — Proof of impact: retrieve the flag

Run a command of your own to demonstrate impact. List the webroot, then read the flag file:

```
# python3 exploit.py -u http://127.0.0.1:8088/ -c 'ls>/out/p1'
PROOF: flag.txt
    index.php

# python3 exploit.py -u http://127.0.0.1:8088/ -c 'cat f*>/out/p1'
PROOF: FLAG{PHP_1SN'T_S4F3_4NYM0R3}
```





Impact confirmed. Arbitrary command execution as the web-server user, flag FLAG{PHP_1SN'T_S4F3_4NYM0R3} recovered — this is a critical, pre-authentication RCE finding.

Detection

The attack leaves distinctive marks. Any one is worth alerting on; the combination is high-confidence:

Signatures

- **Abnormally large PHPSESSID** values in Set-Cookie response headers — far longer than the fixed-length IDs PHP normally issues (the disclosure path).
- **Large multipart POST bodies with very high field counts** carrying the PHP_SESSION_UPLOAD_PROGRESS field (the delivery requests).
- **PHP worker crashes / out-of-memory errors** clustered in the logs (the memory exhaustion that drives the bug).

Remediation

The only complete fix is to patch the PHP runtime itself — update the PHP package or container base image to **8.5.12, 8.4.27, 8.3.36 or 8.2.35**, then restart every worker. Updating WordPress/Laravel/etc. does nothing; the vulnerable code is in PHP, underneath the application.

Interim controls (none a complete substitute for patching):

Interim control	Effect
session.upload_progress.enabled = 0 + restart workers	Removes the trigger for most techniques.
opcache.enable = 0	Frustrates the code-execution path.
fatal_error_backtraces = 0 (PHP 8.5)	Addresses CVE-2026-109619 specifically.
Reject multipart/form-data at the edge	Blocks delivery — breaks legitimate uploads.
Run PHP least-privilege	Limits what a successful chain yields.

A defence that does NOT help

A WAF that runs as a PHP module inspects traffic only once PHP hands control to the application — but this attack does its damage before the application runs. An in-process filter never sees it. Protection must sit in the patched runtime or at a layer in front of PHP.

Learning Objectives — Answers

1. What session.upload_progress does, and why it runs before application code. It reports upload progress by writing the upload's name/size/bytes-received into $_SESSION while PHP parses a multipart body. Body parsing happens before the target script is compiled or run, so the code is reachable on any POST-accepting URL with no login, no session_start() and no application logic.

2. How a memory bailout leaves a surviving dangling pointer. On memory_limit exhaustion PHP longjmp()s out (a bailout), skipping normal cleanup. If the OOM hits after a block is freed but before its pointer is cleared, the stale pointer remains. Under prefork single-worker mod_php the worker serves the next request without restarting, so that pointer survives into the following request.

3. The two CVEs. CVE-2026-109618 is memory disclosure: the stale pointer is read back as an oversized string and returned in the Set-Cookie: PHPSESSID header, leaking up to ~67 KB of worker memory per request. CVE-2026-109619 is RCE: the freed memory is reoccupied with a forged structure whose destructor pointer targets system(), so engine cleanup executes an attacker command. One root cause, two outcomes.

4. Running the chain to leak, defeat ASLR and execute. The public exploit leaks the heap base and the libphp base (two disclosures), which locates the engine despite ASLR; it then drives the second CVE to call system().

Demonstrated in the Walkthrough: id returned uid=33(www-data) and the flag was recovered — end-to-end RCE from multipart POSTs alone.

5. Detection and the mitigations that work. Detect oversized PHPSESSID in Set-Cookie, multipart POSTs carrying PHP_SESSION_UPLOAD_PROGRESS with high field counts, and clustered PHP OOM/worker crashes. Mitigate by patching the runtime (8.5.12 / 8.4.27 / 8.3.36 / 8.2.35) and restarting workers; interim: disable session.upload_progress, opcache.enable=0, reject multipart at the edge, least privilege. A PHP-module WAF cannot see this attack — protection must be in the patched runtime or in front of PHP.